



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Задача коммивояжера

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Алгоритм полного перебора	5
2.2 Функции, реализованные для полного перебора	5
2.3 Муравьиный алгоритм	6
2.4 Функции, реализованные в муравьином алгоритме	6
3 Технологическая часть	8
4 Исследовательская часть	14
Заключение	18
Список используемых источников	19

Введение

Цель лабораторной работы: разработать алгоритм для решения задачи коммивояжера.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать задачу коммивояжера, метод полного перебора и муравьиный алгоритм.
2. Реализовать метод полного перебора.
3. Реализовать муравьиный алгоритм.
4. Провести параметризацию для муравьиного алгоритма.
5. Оценить эффективность нахождения оптимального пути разными алгоритмами.
6. Выполнить тестирование реализации разработанных алгоритмов.

1 Аналитическая часть

Задача коммивояжера [1] — это классическая задача комбинаторной оптимизации, в которой коммивояжер должен посетить каждый из n городов ровно один раз, начиная и заканчивая свой маршрут в одном и том же городе, чтобы минимизировать общую стоимость (или длину) пути. Основная формулировка задачи: у нас есть заданный набор городов и расстояния между каждой парой городов. Коммивояжер должен найти кратчайший путь, который посетит каждый город ровно один раз и вернется в исходный город.

Метод полного перебора в задаче коммивояжера — наиболее простой и точный способ нахождения оптимального решения задачи. Он заключается в переборе всех возможных комбинаций порядка посещения городов и выборе того, который обеспечивает минимальную общую длину пути.

Муравьиный алгоритм — это алгоритм, вдохновленный поведением муравьев при поиске кратчайшего пути к источнику пищи. В контексте задачи коммивояжера муравьиный алгоритм используется для нахождения оптимального пути.

Основная идея заключается в следующем:

1. **Феромоны:** Муравьи откладывают феромоны на ребрах маршрута. Количество феромона на ребре обратно пропорционально длине пути - чем короче путь, тем больше феромона откладывается.
2. **Поиск пути:** Муравьи выбирают следующий город с учетом количества феромона на ребрах и "привлекательности" города.
3. **Обновление феромонов:** После завершения пути муравьи обновляют количество феромона на ребрах в зависимости от длины пройденного ими пути. Краткие и более оптимальные маршруты получают больше феромона.

2 Конструкторская часть

2.1 Алгоритм полного перебора

1. Инициализация начального порядка посещения городов.
 - Создать вектор размером n , где n — количество городов, который содержит последовательность чисел от 1 до n . Каждое число представляет собой порядковый номер города.
2. Перебор всех возможных комбинаций порядка посещения городов.
 - Переставлять элементы вектора так, чтобы они образовывали следующую перестановку, пока все возможные перестановки не будут исчерпаны.
 - Для каждой перестановки выполнять вычисление длины текущего маршрута.
3. Поиск минимальной длины маршрута.
 - Если текущий маршрут короче предыдущего минимального маршрута, обновить значение минимальной длины маршрута и сохранить порядок посещения городов в векторе.
4. Вывод результата.
 - Если найден допустимый маршрут, вывести длину найденного минимального пути и порядок посещения городов, обеспечивающий этот минимальный путь.
 - Если допустимый маршрут не был найден, вывести сообщение о том, что возможного маршрута нет.

2.2 Функции, реализованные для полного перебора

1. Функция “findOptimalPath” — функция, которая ищет путь, на который нужно потратить меньше всего времени и выводит его. Входные данные в функцию: n — количество городов и матрица “graph”, которая содержит затраты по времени на перемещение из одного пункта в другой.
2. Функция “calculatePathLength” — функция, которая считает затраты времени на обход всех городов. Входные данные в функцию: вектор “path”, представляющий порядок посещения городов и матрица смежности “graph” — двумерный вектор, представляющий веса ребер между городами.

2.3 Муравьиный алгоритм

1. Инициализация феромонов.

- Начальные значения феромонов устанавливаются на рёбрах графа. Чем выше количество феромонов, тем более привлекательным считается путь.

2. Проход муравьёв.

- Муравьи перемещаются по графу, строя маршруты от начальной точки до конечной. Они выбирают следующий город, опираясь на феромоны и информацию о расстояниях между городами.

3. Обновление феромонов.

- После прохода всех муравьёв, феромоны обновляются на основе пройденных маршрутов. Чем короче маршрут, тем больше феромона откладывается на его рёбрах.

4. Испарение феромонов.

- Чтобы избежать слишком сильного притяжения к одному пути, феромоны испаряются со временем. Это позволяет алгоритму исследовать разные пути.

5. Повторение процесса.

- Шаги с 2 по 4 повторяются многократно (заданное количество итераций или до достижения условия останова).

6. Выбор лучшего маршрута

- После завершения всех итераций выбирается лучший найденный маршрут, основываясь на количестве феромона и длине пути.

2.4 Функции, реализованные в муравьином алгоритме

1. Функция “initializePheromones” — функция, которая заполняет матрицу “pheromones” начальными значениями феромонов. Это начальное количество феромонов на всех рёбрах между городами. Входные данные в функцию: “pheromones” — ссылка на двумерный вектор, представляющий матрицу феромонов между городами, n — количество городов.
2. Функция “updatePheromones” — функция, которая уменьшает количество феромонов на всех рёбрах, что соответствует их испарению. Обновляет феромоны на рёбрах, пройденных муравьями, путём добавления значения к текущим значениям феромонов на этих рёбрах. Входные данные в функцию: “pheromones” — ссылка на двумерный вектор, представляющий матрицу феромонов между городами и “antsPaths” — матрица, содержащий маршруты, пройденные муравьями.

3. Функция “selectNextCity” — функция, в которой методика выбора следующего города основана на вероятностях. Из вероятностей выбирается случайное число и находится город, вероятность выбора которого превышает это число. Параметры функции: “currCity” — текущий город, из которого муравей делает выбор. “graph” — матрица смежности городов. “probabilities” — вектор вероятностей следующих городов для посещения.
4. Функция “antColonyOptimization”. В этой функции реализован основной процесс муравьиной оптимизации.
- Инициализируются феромоны на рёбрах.
 - Несколько муравьёв создают и проходят свои маршруты.
 - Феромоны обновляются на основе пройденных маршрутов.
 - Лучший найденный маршрут сохраняется и возвращается в качестве результата.

Параметры функции: n — количество городов. “graph” — матрица смежности городов. “maxDays” — максимальное количество итераций алгоритма.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<vector>` — это контейнер в стандартной библиотеке C++, который представляет собой динамический массив, способный изменять свой размер динамически во время выполнения программы. Также была подключена библиотека `<algorithm>`, предоставляющая множество шаблонных функций для работы с контейнерами, итераторами и другими структурами данных. Библиотека `<random>` предоставляет средства для генерации случайных чисел и различных распределений вероятности. Библиотека `<time.h>` в C++ содержит набор функций и типов данных для работы со временем и датой.

Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для реализации задачи коммивояжера

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
#include <time.h>
using namespace std;

const int INF = 1e9;
const double ALPHA = 1.0;
const double BETA = 2.0;
const double EVAPORATION = 0.5;
const double Q = 100.0;
const int ANTS = 10;
const int ITERATIONS = 200;

// Функция для вычисления общей длины маршрута
int calculatePathLength(const vector<int>& path, const vector<vector<int>
>>& graph) {
    int pathLength = 0;
    int n = path.size();
    for (int i = 0; i < n - 1; ++i) {
        if (graph[path[i]][path[i + 1]] == 0) { //
            Проверка наличия ребра между городами
            return INF;
        }
        pathLength += graph[path[i]][path[i + 1]];
    }
}
```



```

        if (graph[path[n - 1]][path[0]] != 0) { //
            Проверка наличия ребра между последним и первым городами
            pathLength += graph[path[n - 1]][path[0]];
            return pathLength;
        }
        return INF;
    }

// Функция для поиска минимального пути методом полного перебора
void findOptimalPath(int n, vector<vector<int>>& graph) {
    vector<int> vertex(n);
    for (int i = 0; i < n; ++i) {
        vertex[i] = i;
    }

    int minPath = INF;
    vector<int> bestPath;

    do {
        int currentPath = calculatePathLength(vertex, graph);
        if (currentPath < minPath) {
            minPath = currentPath;
            bestPath = vertex;
        }
    } while (next_permutation(vertex.begin() + 1, vertex.end()));

    if (minPath == INF) {
        cout << "Нет возможного маршрута " << endl;
    }
    else {
        cout << "Минимальный путь: " << minPath << endl;
        cout << "Лучший маршрут через полный перебор : ";
        for (int i = 0; i < n; ++i) {
            cout << bestPath[i] << " ";
        }
        cout << bestPath[0] << endl;
    }
}

// Функция для инициализации феромонов на ребрах
void initializePheromones(vector<vector<double>>& pheromones, int n) {
    pheromones.assign(n, vector<double>(n, 1.0)); //

```

Начальноеколичествоферомона

}

// Функциядляобновленияферомоновнаребрах

```
void updatePheromones(vector<vector<double>>& pheromones, const vector<
    vector<int>>& antsPaths) {
    int n = pheromones.size();
    // Испарениеферомонанавсехребрах
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            pheromones[i][j] *= (1.0 - EVAPORATION);
        }
    }
    // Обновлениеферомонапомаршрутаммуравьев
    for (const auto& path : antsPaths) {
        int pathLength = calculatePathLength(path, antsPaths);
        for (int i = 0; i < n - 1; ++i) {
            pheromones[path[i]][path[i + 1]] += Q /
                pathLength;
        }
        pheromones[path[n - 1]][path[0]] += Q / pathLength;
    }
}
```

// Функциядлявыбораследующегородамуравья

```
int selectNextCity(int currCity, const vector<vector<int>>& graph, const
    vector<double>& probabilities) {
    double sum = 0.0;
    for (size_t i = 0; i < graph.size(); ++i) {
        if (i != currCity && graph[currCity][i] != 0) {
            sum += probabilities[i];
        }
    }

    double randVal = static_cast<double>(rand()) / RAND_MAX;
    double cumulativeProb = 0.0;
    for (size_t i = 0; i < graph.size(); ++i) {
        if (i != currCity && graph[currCity][i] != 0) {
            cumulativeProb += probabilities[i] / sum;
            if (randVal <= cumulativeProb) {
                return i;
            }
        }
    }
}
```

```

        }

    }

    return -1;
}

// Функция для поиска маршрута муравьиной колонии
vector<int> antColonyOptimization(int n, vector<vector<int>>& graph, int
    maxDays) {
    vector<vector<double>> pheromones(n, vector<double>(n));
    initializePheromones(pheromones, n);

    vector<int> bestPath;
    int bestPathLength = INF;
    int days = 0;

    for (int iter = 0; iter < ITERATIONS && days < maxDays; ++iter) {
        vector<vector<int>> antsPaths(ANTS, vector<int>(n));
        for (int ant = 0; ant < ANTS; ++ant) {
            vector<int> visited(n, 0);
            vector<int> path(n);
            path[0] = rand() % n;
            visited[path[0]] = 1;

            for (int step = 1; step < n; ++step) {
                int currentCity = path[step - 1];
                vector<double> probabilities(n, 0.0);
                double probabilitiesSum = 0.0;

                for (int i = 0; i < n; ++i) {
                    if (i != currentCity && visited[i]
                        == 0 && graph[currentCity][i]
                        != 0) {
                        probabilities[i] = pow(
                            pheromones[currentCity]
                                [i], ALPHA) * pow(1.0
                                    / graph[currentCity][i], BETA);
                        probabilitiesSum +=
                            probabilities[i];
                    }
                }
            }
        }
    }
}

```

```

        if (probabilitiesSum > 0.0) {
            for (int i = 0; i < n; ++i) {
                probabilities[i] /=
                    probabilitiesSum;
            }
            int nextCity = selectNextCity(
                currentCity, graph,
                probabilities);
            path[step] = nextCity;
            visited[nextCity] = 1;
        }
    }
    antsPaths[ant] = path;
}

updatePheromones(pheromones, antsPaths);

for (const auto& path : antsPaths) {
    int pathLength = calculatePathLength(path, graph)
        ;
    if (pathLength < bestPathLength) {
        bestPathLength = pathLength;
        bestPath = path;
    }
}

for (const auto& path : antsPaths) {
    int pathLength = calculatePathLength(path, graph)
        ;
    if (pathLength < bestPathLength) {
        bestPathLength = pathLength;
        bestPath = path;
    }
}

    days++;
}
return bestPath;
}

int main() {
    setlocale(0, "");

```

```

int n;
cout << "Введите количество городов : ";
cin >> n;

vector<vector<int>> graph = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};

int maxDays;
cout << "Введите
    максимальное количество суток на поиск муравьиным алгоритмом : ";
cin >> maxDays;

findOptimalPath(n, graph);
cout << endl;

srand(static_cast<unsigned int>(time(0)));
vector<int> result = antColonyOptimization(n, graph, maxDays);

cout << "Лучший маршрут через муравьиный алгоритм : ";
for (int i = 0; i < n; ++i) {
    cout << result[i] << " ";
}
cout << result[0] << endl;

return 0;
}

```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы.

Результат работы программы, где пользователь вводит максимальное количество суток на поиск муравьями пути. Программа выводит затраченное время на прохождение всех узлов и самый короткий путь. Результат приведён на рис. 1, матрица смежности (1)

$$\begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix} \quad (1)$$

```
Введите максимальное количество суток на поиск муравьиным алгоритмом: 20  
Минимальный путь: 80  
Лучший маршрут через полный перебор: 0 1 3 2 0  
Лучший маршрут через муравьиный алгоритм: 0 1 3 2 0
```

Рис. 1 – Пример №1

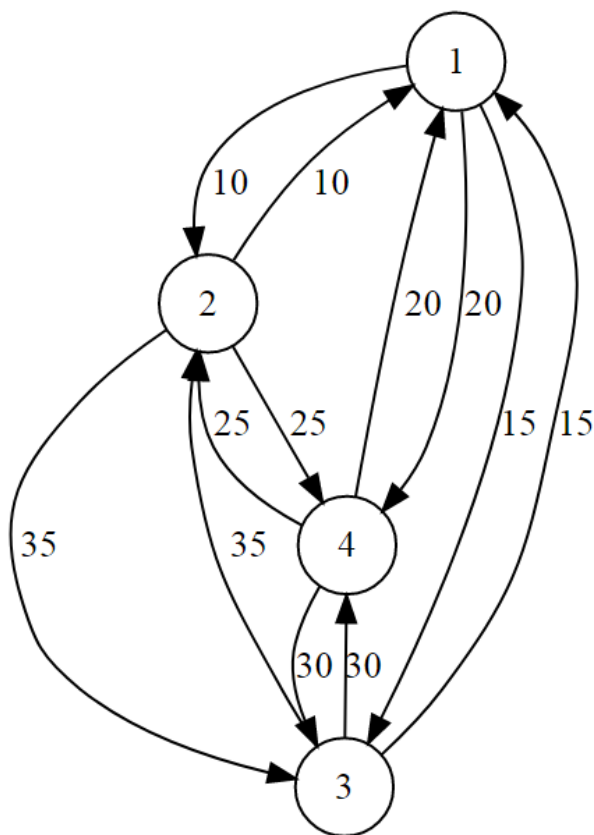


Рис. 2 – Пример графа для теста №1

Результат работы программы, где пользователь вводит максимальное количество суток на поиск муравьями пути. Программа выводит затраченное время на прохождение всех узлов и самый короткий путь. Результат приведён на рис. 3, матрица смежности (2)

$$\begin{pmatrix} 0 & 47 & 12 & 31 & 7 & 35 \\ 6 & 0 & 26 & 27 & 43 & 40 \\ 2 & 49 & 0 & 22 & 35 & 38 \\ 35 & 25 & 50 & 0 & 19 & 29 \\ 19 & 14 & 45 & 13 & 0 & 26 \\ 25 & 49 & 9 & 18 & 20 & 0 \end{pmatrix} \quad (2)$$

```
Введите максимальное количество суток на поиск муравьиным алгоритмом: 10
Минимальный путь: 88
Лучший маршрут через полный перебор: 0 4 1 3 5 2 0
Лучший маршрут через муравьиный алгоритм: 0 4 1 3 5 2 0
```

Рис. 3 – Пример №2

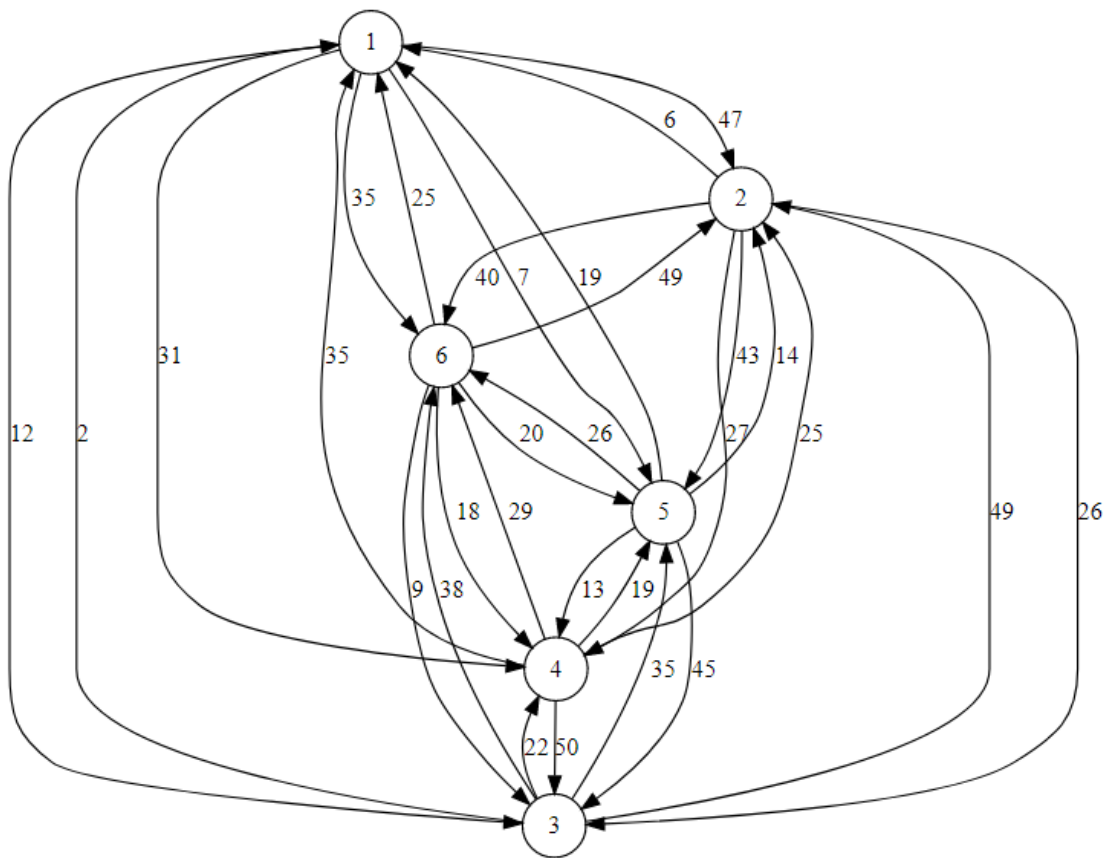


Рис. 4 – Пример графа для теста №2

Результат работы программы, где пользователь вводит максимальное количество суток на поиск муравьями пути. Программа выводит затраченное время на прохождение всех узлов и самый короткий путь. Результат приведён на рис. 5, матрица смежности (3)

$$\begin{pmatrix} 0 & 0 & 9 & 0 & 27 & 0 & 5 & 20 \\ 17 & 0 & 6 & 0 & 0 & 2 & 0 & 0 \\ 0 & 14 & 0 & 3 & 0 & 7 & 0 & 8 \\ 0 & 0 & 0 & 0 & 13 & 0 & 0 & 32 \\ 33 & 0 & 35 & 0 & 0 & 38 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 47 & 0 \\ 0 & 50 & 0 & 52 & 53 & 0 & 0 & 56 \\ 57 & 0 & 0 & 0 & 61 & 0 & 63 & 0 \end{pmatrix} \quad (3)$$

```
Введите максимальное количество суток на поиск муравьиным алгоритмом: 100
Минимальный путь: 250
Лучший маршрут через полный перебор: 0 2 1 5 6 3 7 4 0
Лучший маршрут через муравьиный алгоритм: 2 1 5 6 4 0 7 0 2
```

Рис. 5 – Пример №3

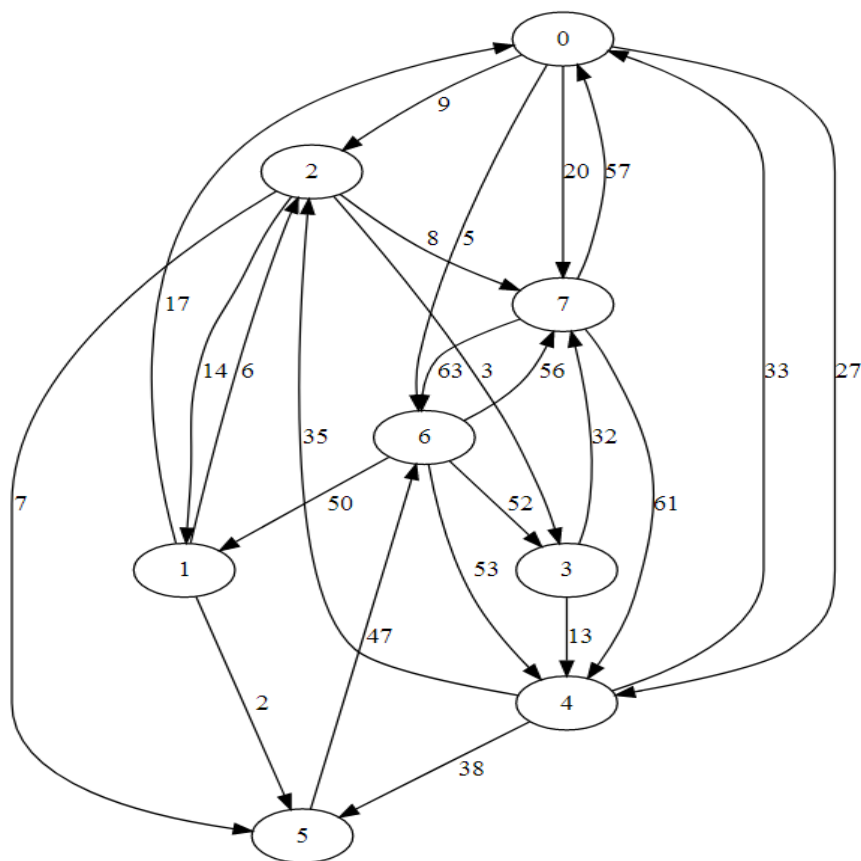


Рис. 6 – Пример графа для теста №3

Плюсы переборного метода:

1. Гарантированное оптимальное решение. Переборный метод обеспечивает нахождение точного оптимального решения, если применен для решения задачи коммивояжера.

Минусы переборного метода:

1. Вычислительная сложность. Время выполнения перебора растет экспоненциально с увеличением количества городов ($O(n!)$). Для больших n количество проверок становится огромным, что делает метод неэффективным и неприменимым на практике.

Плюсы муравьиного алгоритма:

1. Адаптивность к изменениям. Муравьиный алгоритм способен находить приемлемые решения в динамически изменяющихся условиях благодаря своей адаптивной природе.
2. Параллельность и распределенность. Параллелен и применяется для распределенных систем, что делает его эффективным для решения задач на больших данных.
3. Низкая память и вычислительная сложность. Муравьиный алгоритм обладает невысокими требованиями к памяти и умеренной вычислительной сложностью, что позволяет применять его для решения сложных задач.

Минусы муравьиного алгоритма:

1. Склонность к локальным минимумам. Иногда может застрять в локальных оптимумах, не находя глобально оптимального решения.
2. Чувствительность к параметрам. Работоспособность алгоритма сильно зависит от настройки параметров, таких как коэффициенты испарения феромонов, параметры муравьиной информации и т. д.
3. Неприменимость для точного решения. Несмотря на глобальную оптимизацию, муравьиный алгоритм не гарантирует нахождения точного глобального оптимума.

Заключение

Цель лабораторной работы достигнута: разработан алгоритм для решения задачи коммивояжера.

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описана задача коммивояжера, метод полного перебора и муравьиный алгоритм.
2. Реализован метод полного перебора.
3. Реализован муравьиный алгоритм.
4. Проведена параметризация для муравьиного алгоритма.
5. Оценена эффективность нахождения оптимального пути разными алгоритмами.
6. Выполнено тестирование реализации разработанных алгоритмов.

Список литературы

1. Мудров В.И. Задача о коммивояжере (2-е издание). — УРСС, 2013. — 64 с.